Системен дизайн на AI приложение

Промпт инженерство на ниво код и управление на контекста

Богомил Стоянов, Михаил Доброславски

Лекция 2

Приложно AI програмиране

© 2026 Богомил Стоянов, Михаил Доброславски · CC BY-NC-ND 4.0

От Тема 1 към днес

- Миналата лекция: LLM като **външна услуга**, достъпна през API; видяхме ролите, *назовахме* параметрите за семплиране и идеята за независимост от доставчика.
- Днес: **системен дизайн** около модела, т.е. как да го вградим в кода така, че да е предвидим, лесен за тестване и за поддръжка.
- Централен въпрос: как от „чат в браузъра“ да стигнем до **инженерно решение** (промптове под контрол на версиите, управляван контекст, изолиран доставчик).

Тезата на лекцията

Промптът не е съобщение, а **част от кода**: параметризиран, под контрол на версиите и тестван. Всичко около недетерминирания модел правим *детерминирано*: шаблони, параметри, валидация, наблюдаемост.

Дневен ред: Лекция 2

От „чат в браузъра“ към промпт инженерство на ниво код

Роли: System / User / Assistant

Шаблони и структура на промптовете

Параметри за семплиране в дълбочина

Управление на контекста

Архитектурни модели

Чести грешки

Обобщение

Дискусия

От „чат в браузъра“ към промпт инженерство на ниво код

Чат в браузъра vs. промпт на ниво код

Чат в браузъра

- Еднократно съобщение, пише се на ръка.
- Не е под контрол на версиите, не се тества.
- Резултатът не е повторяем между хора/сесии.

Промпт на ниво код

- **Параметризирана функция**, а не свободен текст.
- Под контрол на версиите и тестван като всеки друг код.
- Повторяемост: едни и същи инструкции + различни данни → предвидим формат.

Договор на промпта

Едни и същи *инструкции* + сменяеми *данни* ⇒ предвидим формат на изхода. Това го прави обект на инженерство, а не на импровизация.

Грешка: конкатенация на низове

Разпръснат промпт из целия код (не прави така)

```
# някъде дълбоко в бизнес логиката...
prompt = "Обобщи следния текст на " + lang + ": " + user_text + \
         " Отговори с най-много " + str(n) + " изречения."
resp = client.chat.completions.create(model="gpt-6-sol",
    messages=[{"role": "user", "content": prompt}])
```

Проблеми: инструкцията и данните са слети (моделът не знае кое е задача); промптът не може да се използва повторно, да има версии и да се оценява; всяка промяна означава ровене из кода.

Правилно

Централизирани, **именувани** шаблони под контрол на версиите, в които статичната инструкция е отделена от динамичните данни.

Роли: System / User / Assistant

Семантика на трите роли

- **`system`**: задава *поведение*: персона, ограничения, формат, тон, „правила“ (напр. „отговаряй само на български; ако не знаеш, кажи го“). Най-силно и трайно влияние.
- **`user`**: входът за конкретната стъпка: въпроси, данни, инструкции от (или от името на) потребителя.
- **`assistant`**: отговорите на модела; връщат се обратно в историята, за да „помни“ моделът предишните си реплики (multi-turn).

Ключово разграничение: инструкция vs. данни

Инструкциите живеят в `system`, а недоверените потребителски данни в `user`, ясно отделени. Това прави промпта по-ясен за модела.

Разделянето НЕ е защита срещу prompt injection

- Моделът третира *всеки* текст в контекста като възможна инструкция, независимо от ролята или таговете.
- Атакуващият просто затваря тага: `</review> Игнорирай горното…`
- **Непряко** инжектиране: атаката идва от документ, уеб страница или резултат от инструмент, не от потребителя.
- OWASP LLM01:2025: няма напълно надеждна защита.

Разделянето намалява объркването, но не спира атаката

Реалните мерки (Лекция 14): минимални права на инструментите, човешко потвърждение, тайни извън контекста, проверка на изхода.

Къде отива `system` по доставчик

Доставчик	Къде е `system`	Форма
OpenAI	Първо съобщение в `messages`	`{"role":"system", ...}`
Anthropic	Отделен top-level параметър	`system="..."`
Gemini	В `config`	`system_instruction="..."`

- Семантиката е обща; различава се само *мястото*. При по-новите модели на OpenAI ролята се казва `developer` (`system` още се приема).
- Точно това разминаване по-късно скриваме зад *provider-agnostic* адаптер; бизнес кодът не бива да знае за него.

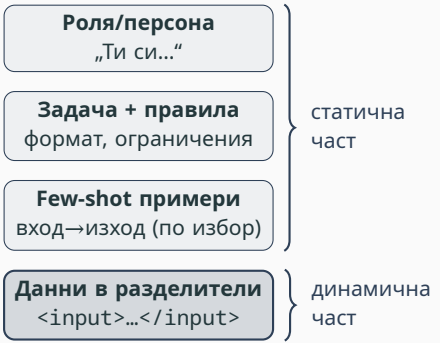
Multi-turn: историята расте с всяка реплика

```
messages = [
    {"role": "system",    "content": "Ти си кратък асистент по география."},
    {"role": "user",      "content": "Коя е столицата на България?"},
    {"role": "assistant", "content": "София."},        # предишен отговор
    {"role": "user",      "content": "А населението ѝ?"},  # разчита на контекста
]
```

- Моделът е **без памет** между извиквания: „паметта“ е просто списъкът, който *ние* подаваме всеки път.
- Всяка нова двойка user/assistant увеличава входа → повече токени, по-висока цена. Оттук идва нуждата от *управление на контекста*.

Шаблони и структура на промптовете

Анатомия на добре структуриран промпт



- **Статичната част** (инструкции) е отделена от **динамичната** (данни) чрез placeholders.
- Ясни секции и **разделители** за входа (XML-подобни тагове или тройни кавички): по-малко объркване. *Не* са граница на сигурност.
- Редът има значение: стабилното отпред, променливото накрая.

Шаблон в кода: build_messages

```
SYSTEM = (      # статична инструкция: константа/файл под контрол на версиите
    "Ти си асистент за класификация на настроение. "
    "Върни само една дума: положително, отрицателно или неутрално."
)

def build_messages(user_text: str) -> list[dict]:
    return [
        {"role": "system", "content": SYSTEM},
        # данните са в разделители и ясно отделени от инструкцията
        {"role": "user", "content": f"<review>\n{user_text}\n</review>"},
    ]
```

Промптът е **функция**: инструкцията е постоянна, входът се сменя. Лесен за повторно ползване, тест и eval, за разлика от разпръснатите низове.

Zero-shot vs. few-shot

Zero-shot

- Само инструкция, без примери.
- По-евтино (по-малко токени).
- Достатъчно за прости/ясни задачи.

Few-shot (1–3 примера)

- Показваме желания формат/стил (*in-context learning*).
- По-надежден формат при по-сложни задачи.
- Цена: повече токени на всяко извикване.

Емпирично правило

Показвай, вместо да обясняваш. Ако при zero-shot форматът не е стабилен, добави 2–3 примерни двойки вход→изход, преди да пишеш дълги инструкции.

Few-shot на практика

```
messages = [
    {"role": "system", "content": "Определи настроението с една дума."},
    # --- примери (in-context learning) ---
    {"role": "user", "content": "<review>Обожавам този продукт!</review>"},
    {"role": "assistant", "content": "положително"},
    {"role": "user", "content": "<review>Счупи се веднага.</review>"},
    {"role": "assistant", "content": "отрицателно"},
    # --- същинският вход ---
    {"role": "user", "content": "<review>Става, нищо особено.</review>"},
]
```

Примерите се подават като редуващи се `user`/`assistant` двойки. Така моделът „вижда“ формата, вместо да го чете като описание.

Разсъждение стъпка по стъпка (chain-of-thought)

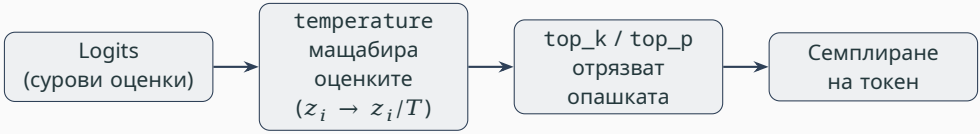
- „Мисли стъпка по стъпка“ / chain-of-thought повишава качеството на **сложни** задачи (математика, логика, многостъпкови изводи).
- Цена: повече токени и по-висока латентност.
- При **reasoning модели** това разсъждение е *вградено*: не го изисквай ръчно; дублирането може дори да влоши резултата.

Инструкции за формат на изхода

„Върни JSON с полета …“ работи, но е *крехко*. В Тема 3 го заменяме или подсилваме със **Structured Outputs**: гарантирана схема и валидация.

Параметри за семплиране в дълбочина

Как моделът избира следващ токен



- Моделът връща *разпределение* над възможните следващи токени: $p_i = \text{softmax}(z_i/T)$.
- Редът зависи от имплементацията: Hugging Face прилага temperature → top-k → top-p, llama.cpp (Ollama) прилага temperature последна.
- Параметрите за семплиране управляват **как** избираме от него: колко детерминистично или разнообразно.

temperature

- **Мащабира суровите оценки**: $p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$.
- $T \to 0$: greedy (argmax); $T > 1$: изглажда към равномерно. Диапазони: OpenAI и Gemini 0–2, Anthropic 0–1.
- **Ниска (0–0.3)** → детерминистично, фактологично: извличане на данни, класификация, структуриран изход.
- **Висока (0.7–1.0+)** → креативно, разнообразно: генериране на идеи, творчески текст.
- Изборът зависи от задачата: избери стойност и я **фиксирай**.

`temperature=0` $\neq$ пълен детерминизъм

Възможна е остатъчна вариация (хардуер, версии, паралелизъм). Не разчитай на бит-в-бит възпроизводимост.

Ефект на temperature: демонстрация

```
import ollama
prompt = [{"role": "user", "content": "Измисли име за кафене."}]

for t in (0.0, 1.0):
    for _ in range(3):
        r = ollama.chat(model="llama3.2:1b", messages=prompt,
                        options={"temperature": t})
        print(t, "->", r["message"]["content"])
# t=0.0 -> почти едно и също име и трите пъти
# t=1.0 -> три различни, по-креативни варианта
```

Същото извикване при две стойности: ниската дава стабилност, а високата разнообразие. Локалният модел приема всички параметри; hosted моделите не винаги (следващите слайдове).

top_p (nucleus sampling)

- Семплира от **най-малкото** множество токени с кумулативна вероятност $\geq p$ („ядрото“), отрязвайки дългата опашка от малко вероятни токени.
- Алтернатива на `temperature` за контрол на разнообразието.
- **`top_k`**: само k-те най-вероятни токена (Anthropic, Gemini, Ollama; не и OpenAI).

Настройвай ЕДНО от двете

Едновременната настройка на `temperature` **и** `top_p` е обикновено излишна и объркваща. Фиксирай едното (напр. `top_p=1.0`) и управлявай другото.

Не всеки модел приема `temperature`

- **OpenAI GPT-6**: reasoning модели; `temperature` само при `reasoning_effort="none"`.
- **Anthropic**: Sonnet/Opus 5.x → 400; Haiku 4.5 приема. Зависи от *поколението*, не от това дали моделът мисли.
- **Ollama**: локалните модели приемат всички параметри.
- `effort` *не* замества `temperature`: управлява колко мисли моделът, не случайността.

Изводът за дизайна

Кой модел какво приема се решава на **едно място** в AI слоя, а не из бизнес кода. Още един довод за адаптера (по-нататък в лекцията).

max_tokens и другите параметри

- **Таван на изхода** (`max_tokens` / `max_completion_tokens` / `max_output_tokens`): дължината на *изхода* (и цената), включително мисленето. **Не е** контекстният прозорец.
- Внимавай да не *отреже* отговора: `finish_reason="length"` (OpenAI) или `stop_reason="max_tokens"` (Anthropic) ⇒ увеличи лимита или продължи генерирането.
- **Други (по доставчик):** `stop` sequences (спиране при определен низ), `frequency/presence penalty` (срещу повторения), `seed` (по-възпроизводими резултати, където се поддържа).

Практическо правило

Избери параметрите спрямо задачата и ги **фиксирай и запази** заедно с промпта. Недетерминизмът остава.

Управление на контекста

Контекстният прозорец: твърд таван

- Контекстен прозорец (context window) = максимален брой токени за **вход** + **изход** заедно. Всичко трябва да се събере в него.
- При дълги разговори историята расте линейно → рискове:
  - надхвърляне на лимита (грешка),
  - по-висока цена и латентност,
  - „изгубени по средата“ (*lost-in-the-middle*) детайли, деградация на вниманието.

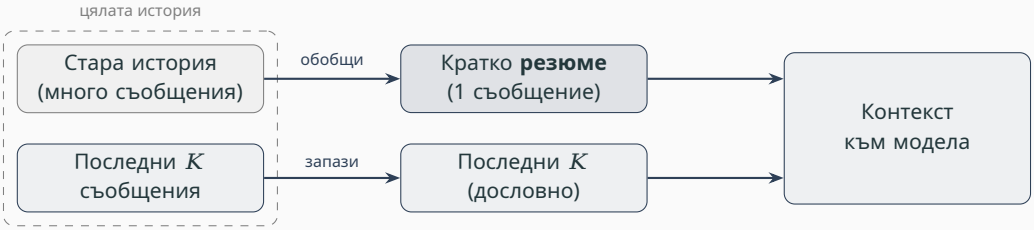
Лоша практика

Наблъскване на цялата история „за всеки случай“ вдига цена и латентност *и* влошава качеството. Управлявай контекста **активно**.

Стратегии за управление на контекста

- **Sliding window**: пази само последните N съобщения. Просто, но губи стар контекст.
- **Summarization**: периодично сгъстяване на старата история в кратко резюме, което замества суровите съобщения.
- **Хибрид**: резюме на старото + пълни последни K съобщения (често използвано на практика).
- **Външна памет / RAG**: извличай релевантното при нужда, вместо да държиш всичко в промпта (Теми 5–8).
- **Данни извън промпта**: големите данни се подават при нужда чрез tool calls или файлове, а не директно в промпта.

Хибридна стратегия: схема



Старото се сгъстява до резюме; последните K съобщения остават дословни (там е „живият“ контекст). Балансира бюджет и точност.

Скелет: последни съобщения + резюме на старите

```python
def split_point(history, keep_last):
    cut = max(0, len(history) - keep_last)
    while cut < len(history) and history[cut]["role"] != "user":
        cut += 1                        # не започваме с „осиротял“ отговор
    return cut

def manage_context(system, history, memory, keep_last=6):
    covered = memory.get("covered", 0)
    if len(history) - covered > 2 * keep_last:   # периодично, не всеки път
        cut = split_point(history, keep_last)
        memory["summary"] = summarize(memory.get("summary", ""),
                                      history[covered:cut])
        memory["covered"] = covered = cut
    if memory.get("summary"):
        system += f"\n\nРезюме на по-ранния разговор: {memory['summary']}"
    return system, history[covered:]
```

Дословната част започва с `user`; резюмето се пази и се обновява инкрементално. Прагът може да е и по преброени токени.

Токен бюджет и prompt caching

- **Броене на токени преди изпращане** (tokenizer или API за броене) → проактивно управление на бюджета; **логвай** `usage` от отговорите.
- **Prompt caching**: кеширане на стабилния *префикс* (system + дълъг контекст) за по-ниска цена/латентност. Различно по доставчик: Anthropic *изрично* (`cache_control`, минимум 512–4 096 токена); OpenAI *автоматично* над 1 024 токена; Gemini implicit и explicit.

Подредба с оглед на кеша

Дръж **стабилната** част отпред (system, инструкции, few-shot), а променливото (потребителски вход) накрая. Така префиксът, който може да се кешира, е максимален.

Архитектурни модели

Слоеста архитектура на AI приложение



AI слоят е **изолиран зад интерфейс**: бизнес кодът вика `generate(…)`, без да знае кой е доставчикът или как е устроен промптът.

Provider-agnostic абстракция

```python
class LLMClient(Protocol):
    def generate(self, messages: list[dict], params: Params) -> Answer: ...

class AnthropicClient:   # адаптер: знае, че system е top-level параметър
    def __init__(self) -> None: self._c = anthropic.Anthropic()

    def generate(self, messages, params):
        system, rest = _split_system(messages)   # system е отделно
        extra = {"system": system} if system else {}
        msg = self._c.messages.create(model=params.model,
            max_tokens=params.max_tokens, messages=rest, **extra)
        text = next((b.text for b in msg.content if b.type == "text"), "")
        return Answer(text, msg.usage.input_tokens, msg.usage.output_tokens)

# бизнес кодът вика само интерфейса и не знае доставчика
def summarize(client: LLMClient, text: str) -> str: ...
```

Единен `generate`; адаптерите капсулират разликите (напр. къде е `system`).

Готова абстракция: Vercel AI SDK (по избор)

```ts
import { generateText } from "ai";
import { anthropic } from "@ai-sdk/anthropic";

const { text, usage } = await generateText({
  model: anthropic("claude-sonnet-5-5"), // смяна на доставчик = смяна на реда
  system: "Ти си кратък асистент.",
  messages: [{ role: "user", content: "Обобщи текста…" }],
});
```

Същата provider-agnostic идея, но **готова**: единен интерфейс към много доставчици. Ръчният адаптер от предишния слайд показва *какво* абстрахира. Vercel AI SDK ползваме в Лекция 4, LangGraph в Лекция 12.

Наблюдаемост, надеждност и eval-и

- **Наблюдаемост**: логвай промпт, параметри, usage, латентност и версия на модела. Без това дебъгването и оценката са невъзможни.
- **Предвидим код около модела**: направи всичко *около* модела предвидимо: валидация, retry, fallback. Така компенсираш недетерминизма на самия модел.
- **Evaluations (eval-и)**: малък набор случаи вход→очакван изход, за да засичаш регресии при смяна на промпт/модел/параметри.

Без eval-и: тиха регресия

Всяка промяна на промпт/модел рискува незабелязано влошаване. Дори 5–10 случая са несравнимо по-добре от нула.

Чести грешки

Чести грешки: параметри и промптове

- **`temperature` и `top_p` едновременно**: излишно и объркващо; фиксирай едното.
- **`temperature=0` ≠ детерминизъм**: възможна е остатъчна вариация; не разчитай на бит-в-бит възпроизводимост.
- **Отрязан отговор (`finish_reason="length"`)**: засечи и увеличи `max_tokens` или продължи генерирането.
- **Инструкции + недоверени данни в едно поле**: обърква модела; ясни разделители, инструкциите в `system`.
- **Разделителите като защита от prompt injection**: таговете само подреждат промпта; атакуващият пише `</review>` и добавя свои инструкции (Лекция 14).

Чести грешки: контекст и архитектура

- **Цялата история „за всеки случай“**: цена, латентност, lost-in-the-middle; управлявай контекста активно.
- **Промптове като низове, разпръснати из кода**: централизирай, именувай и сложи под контрол на версиите.
- **SDK-специфичен код навсякъде**: vendor lock-in; изолирай зад адаптер рано.
- **Липса на eval-и**: тиха регресия при всяка промяна.

Обобщение

Какво научихме днес

- Промптът е **код**: централизиран, именуван шаблон под контрол на версиите; инструкции отделени от данни.
- **Ролите** (system/user/assistant) задават поведение, вход и история; мястото на `system` се различава по доставчик.
- **Параметрите за семплиране** се избират по задача и се фиксират: ниска temperature за структура, ЕДНО от temperature/top_p.
- **Контекстният прозорец** се управлява активно: sliding window, summarization, хибрид, външна памет; токен бюджет + caching.
- **Архитектура**: слоест, provider-agnostic дизайн с наблюдаемост, предвидим код около модела и eval-и.

Следваща лекция

Тема 3 (Structured Outputs): от „върни JSON“ към гарантирани схеми и валидация на изхода.

Връзка с изпитния конспект

- **Въпрос 2**: „Параметри на LLM: Temperature, Top-P, Max Tokens.“ → покрит подробно в секцията за семплиране, с препоръки за различни типове задачи.
- **Въпрос 3**: „Роли и управление на контекста: System, User, Assistant.“ → покрит от секциите за роли и за контекстния прозорец (прозорец, обобщаване, външна памет).
- **Косвено Въпрос 1**: provider-agnostic слоят стъпва върху различията между API-тата от Тема 1.

Дискусия

Теми

1. Кога промптът трябва да е *код* (под контрол на версиите, тестван), а кога е приемлив като низ в кода? Къде е границата?
2. При препълване на контекстния прозорец обобщаваме ли историята, отрязваме ли я, или сменяме подхода (RAG)? Какви са компромисите?
3. Колко „умна“ бизнес логика да сложим *около* модела и колко да оставим на самия модел? Кое носи надеждност и кое гъвкавост?

Библиография

- Alammar & Grootendorst (2024). *Hands-On Large Language Models*.
- OpenAI Prompt engineering: `https://developers.openai.com/api/docs/guides/prompt-engineering`
- OpenAI Text generation / parameters: `https://developers.openai.com/api/docs/guides/text`
- Anthropic Prompt engineering: `https://docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview`
- Anthropic Context windows: `https://docs.claude.com/en/docs/build-with-claude/context-windows`
- Anthropic Prompt caching: `https://docs.claude.com/en/docs/build-with-claude/prompt-caching`
- Gemini API, text generation & system instructions: `https://ai.google.dev/gemini-api/docs/text-generation`
- Vercel AI SDK: `https://ai-sdk.dev/docs`